KU LEUVEN

Verslag geavanceerde computerarchitectuur

Sessie 1: Getting Acquainted with GPU Programming

Daan Vercammen
Klaas Meersman

ELICT: ELECFS
20 februari 2025
Master in Engineering Technology
Academic year 2024-2025

1.1 Probleemstelling en doel

Het doel van de opdracht is om kennis te maken met CUDA en het programmeren van een GPU. We onderzoeken het verschil in uitvoeringssnelheid tussen een GPU en een CPU. Dit doen we aan de hand van een array die 'omgekeerd' moet worden. Deze taak kan geparallelliseerd worden op een GPU omdat er geen afhankelijkheden zijn tussen de elementen. We verwachten dat de grootte van de array invloed heeft op welke methode efficiënter is, dus zullen we verschillende arraygroottes testen.

1.2 Methodologie

We definiëren een `global` functie `reverseArrayGPU` (appendix A, lijn 14) die gecalled wordt vanuit de CPU en uitgevoerd wordt op de GPU. Deze functie dient dus het omdraaien van de array uit te voeren op de GPU. We vergelijken deze uitvoering met de uitvoering op de CPU, `reverseArrayCPU` (appendix A, lijn 25). De uitvoertijd van zowel de CPU- als de GPU-functie worden gemeten voor verschillende groottes van arrays. Merk op dat we onderscheid maken tussen de uitvoertijd voor de GPU-kernel en de totale GPU-tijd (met geheugenallocatie).

1.3 Resultaten en observaties

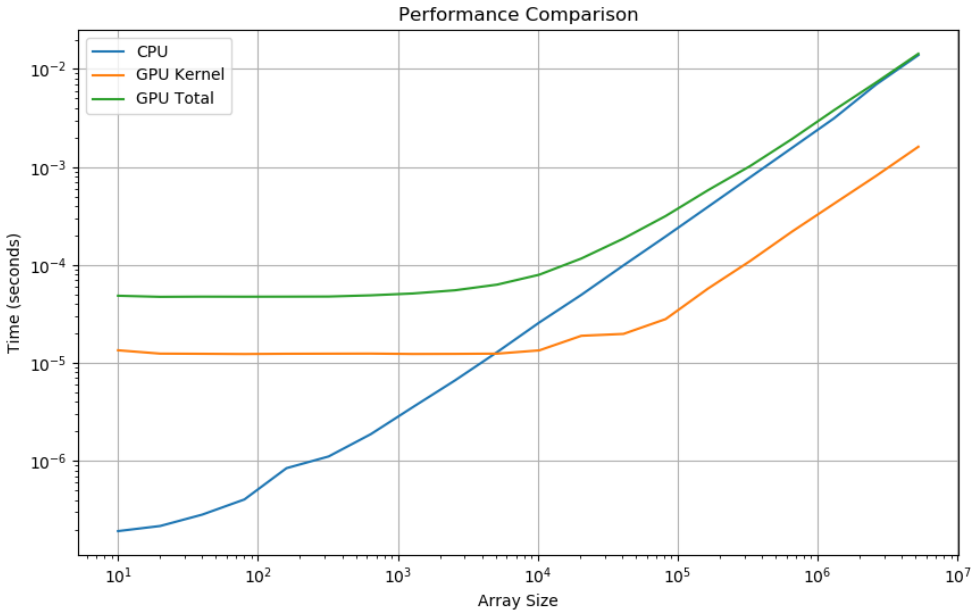
De resultaten (Figuur 1.1) zullen in deze sectie besproken worden. We merken hierbij enkele zaken op.

1.3.1 Kleine array

Bij een kleine array presteert een CPU veel beter dan een GPU. Dit heeft een aantal oorzaken, eerst en vooral omdat er relatief veel tijd verloren gaat bij het alloceren en vullen van het geheugen in de GPU. Maar zelfs de kerneltijd is groter in de GPU. Dit kan verklaard worden doordat voor sequentiële taken een individuele GPU-core doorgaans trager is dan een CPU-core.

1.3.2 Middelgrote array

Pas bij een middelgrote array zal de GPU-kerneloperatie sneller worden dan de CPU. Door de geheugenoverdracht zal de totale GPU-operatie echter nooit sneller worden. Verder merken we op dat de GPU-kerneltijd tot bij een arraygrootte van 10^4 stabiel bleef. Grotere arrays lijken echter ook exponentieel meer tijd nodig te hebben. We vermoeden dat hier de



Figuur 1.1: Uitvoertijden voor verschillende arraygroottes (logaritmische schaal)

beschikbare GPU-cores volledig benut worden, waardoor verdere parallellisatie niet meer mogelijk is en ook serialisatie optreedt.

1.3.3 Grote array

We merken op dat bij grote arrays de totale GPU-tijd asymptotisch nadert naar de CPU-tijd. In dit geval is de parallellisatie zodanig ver doorgedreven dat enkel de schrijfsnelheid er nog toe doet. Het serieel opvullen van de geheugenlocaties in de GPU wordt de beperkende factor, vergelijkbaar met het seriële schrijven in de array van de CPU. De kerneloperaties worden in dit geval verwaarloosbaar. Ze zijn immers ongeveer een factor duizend kleiner dan de totale GPU-tijd.

1.3.4 Conclusie

In dit labo leerden we werken met CUDA en GPU-programmeren. We onderzochten de uitvoeringssnelheid in functie van de arraygrootte. Zo ontdekten we dat de CPU sneller was bij kleine arrays. Maar bij grote arrays begint het verschil steeds kleiner te worden. We merken op dat er weinig tot geen tijdswinst wordt geboekt met een GPU als de operaties relatief eenvoudig zijn, zoals in dit geval van array-omkering. Enkel als er veel operaties uitgevoerd worden op dat geheugen, zal parallellisatie nuttig zijn. Het is dus beter om heel eenvoudige berekeningen op de CPU uit te voeren.

A CODE

```
#include <cuda_runtime.h>
#include <stdio.h>
#include <chrono>
#include <iostream>
#include <random>
#include <vector>

struct GPUExecutionTimes {
    float kernelTime;
    float totalTime;
};


__global__ void reverseArrayGPU(int *d_array, int size) {
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    int swap_idx = size - 1 - idx;

    if (idx < size / 2) {
        int temp = d_array[idx];
        d_array[idx] = d_array[swap_idx];
        d_array[swap_idx] = temp;
    }
}

void reverseArrayCPU(int *d_array, int size) {
    for(int idx = 0; idx < size/2; idx++){
        int swap_idx = size - 1 - idx;

        int temp = d_array[idx];
        d_array[idx] = d_array[swap_idx];
        d_array[swap_idx] = temp;
    }
}


float runCPU(int array[], int size){
    //CPU
    //measuring
    const auto start = std::chrono::steady_clock::now();
    //measuring

    reverseArrayCPU(array, size);

    //measuring
    const auto end = std::chrono::steady_clock::now();
    const std::chrono::duration<double> elapsed_seconds{end-start};
    //measuring ;

    return elapsed_seconds.count();
```

```
}

GPUExecutionTimes runGPU(int array[], int size){
    int *GPU_d_array;
    cudaMalloc((void**)&GPU_d_array, size * sizeof(int));
    int threadsPerBlock = 1024;
    int blocksPerGrid = (size + threadsPerBlock - 1) / threadsPerBlock;

    // Measure total time including data transfer
    cudaEvent_t start_cuda_total, stop_cuda_total;
    cudaEventCreate(&start_cuda_total);
    cudaEventCreate(&stop_cuda_total);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_total);

    // Copy data from host to device
    cudaMemcpy(GPU_d_array, array, size * sizeof(int), cudaMemcpyHostToDevice);

    // Measure kernel execution time separately
    cudaEvent_t start_cuda_kernel, stop_cuda_kernel;
    cudaEventCreate(&start_cuda_kernel);
    cudaEventCreate(&stop_cuda_kernel);
    cudaDeviceSynchronize();
    cudaEventRecord(start_cuda_kernel);

    // Launch kernel
    reverseArrayGPU<<<blocksPerGrid, threadsPerBlock>>>(GPU_d_array, size);

    // Measure kernel execution time
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_kernel);
    cudaEventSynchronize(stop_cuda_kernel);
    float ms_kernel;
    cudaEventElapsedTime(&ms_kernel, start_cuda_kernel, stop_cuda_kernel);

    // Copy result back to host
    cudaMemcpy(array, GPU_d_array, size * sizeof(int), cudaMemcpyDeviceToHost);

    // Measure total time including data transfer
    cudaDeviceSynchronize();
    cudaEventRecord(stop_cuda_total);
    cudaEventSynchronize(stop_cuda_total);
    float ms_total;
    cudaEventElapsedTime(&ms_total, start_cuda_total, stop_cuda_total);

    // Free device memory
    cudaFree(GPU_d_array);
    GPUExecutionTimes times;
    times.kernelTime = ms_kernel / 1000.0; // Convert to seconds
    times.totalTime = ms_total / 1000.0;   // Convert to seconds
    return times;
}




// Function to generate a random array
int* generateRandomArray(size_t size) {
    int* array = new int[size];

```

```
    std::mt19937 gen(static_cast<unsigned int>(std::time(nullptr)));
    std::uniform_int_distribution<> dis(0, 999);

    for (size_t i = 0; i < size; ++i) {
        array[i] = dis(gen);
    }

    return array;
}

int main() {
    for(int size = 10; size<10000000;size*=2){
        const int amountOfRunsPerSize = 100;


        int* h_array = generateRandomArray(size);

        float averageGPUKernelTime = 0.0f;
        float averageGPUTotalTime = 0.0f;
        float averageCPUTime = 0.0f;
        float cpuTime;


        for(int i = 0;i<amountOfRunsPerSize;i++){
            GPUExecutionTimes gpuTimes = runGPU(h_array, size);
            cpuTime = runCPU(h_array,size);

            averageGPUKernelTime += gpuTimes.kernelTime;
            averageGPUTotalTime += gpuTimes.totalTime;
            averageCPUTime += cpuTime;
        }
        averageGPUKernelTime = averageGPUKernelTime/amountOfRunsPerSize;
        averageGPUTotalTime = averageGPUTotalTime/amountOfRunsPerSize;
        averageCPUTime = averageCPUTime/amountOfRunsPerSize;


        //printf("Array size: %d\n", size);
        //printf("Elapsed time CPU: %.6e seconds\n", averageCPUTime);
        //printf("Elapsed time GPU (kernel only): %.6e seconds\n", averageGPUKernelTime);
        //printf("Elapsed time GPU (total, including data transfer): %.6e seconds\n",
    averageGPUTotalTime);


        printf("%d,%.6e,%.6e,%.6e\n", size, averageCPUTime, averageGPUKernelTime,
    averageGPUTotalTime);

        delete[] h_array;
    }

    return 0;
}
```

REFERENTIELIJST

[1] *CUDA: Determining threads per block, blocks per grid*. Stack Overflow. Geraardpleegd op 17 februari 2025. 2010. URL: https://stackoverflow.com/questions/4391162/cuda-determining-threads-per-block-blocks-per-grid.

[2] NVIDIA Corporation. *CUDA C++ Programming Guide*. Geraadpleegd op 13 februari 2025. NVIDIA. verschillend. URL: https://docs.nvidia.com/cuda/cuda-c-programming-guide/.